LAPORAN PRAKTIKUM
ALGORITMA DAN STRUKTUR DATA

Dosen Pengampu: Muhammad Reza Zulman,S.ST.,M.Sc

Di susun oleh :

Nama	: Wahyu Hakim
NIM	: 2025573010096
Kelas	: TI. 1C
Mata Kuliah	: Algoritma dan Struktur Data

PROGRAM STUDI INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2026/2027

LEMBAR PENGESAHAN

Nama Praktikum	:	Praktikum *Function* dan *Array JavaScript*
Nama	:	Wahyu Hakim
NIM	:	2025573010096
Kelas	:	TI 1C
Jurusan	:	Teknologi Informasi dan Komputer
Prodi	:	Teknik Informatika
Tanggal Praktikum	:	8 April 2026
Tanggal Penyerahan Laporan	:	14 April 2026
Nilai	:	
Keterangan	:	

Buket Rata, 14 April 2026

Muhammad Reza Zulman, S.ST.,M.Sc

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh

Puji syukur saya panjatkan kehadirat Tuhan Yang Maha Esa yang telah memberikan rahmat dan karunia-Nya sehingga saya dapat menyelesaikan laporan dengan judul “Implementasi Function dan Array dalam JavaScript”. Laporan yang sudah saya susun dengan sistematis dan sebaik mungkin ini bertujuan untuk memenuhi tugas mata kuliah Sistem Operasi.

Dengan terselesainya laporan ini, maka tidak lupa saya mengucapkan banyak terima kasih kepada semua pihak yang terlibat dalam penyusunan laporan ini, khususnya kepada Bapak Muhammad Reza Zulman, S.ST.,M.Sc. selaku Dosen Pengampu.

Demikian laporan yang saya buat, mohon kritik dan sarannya atas kekurangan dalam penyusunan laporan ini. Semoga laporan ini dapat bermanfaat bagi semua pihak dan bagi saya selaku penulis.

Lhokseumawe, 14 Maret 2026

Wahyu Hakim

(2025573010096)

DAFTAR ISI

LEMBAR PENGESAHAN ……i

KATA PENGANTAR ……...ii

DAFTAR ISI …….iii

DAFTAR ISI …….iv

BAB I: PENDAHULUAN …….1

1.1 Latar Belakang ……..1

1.2 Rumusan Masalah …….1

1.3 Tujuan Praktikum……... 2

BAB II: DASAR TEORI……3

2.1 Function dalam JavaScript……..3

2.2 Array dalam JavaScript……..3

2.3 Callback dan Arrow Function……4

2.4 Rekursi……4

BAB III: METODE MENGINSTAL……6

3.1 Waktu dan Tempat…….6

3.2 Alat dan Bahan……6

3.3 Prosedur Praktikum……6

BAB IV: HASIL DAN PEMBAHASAN……31

4.1 Hasil praktikum……31

4.1 Pembahasan……..31

BAB V: PENUTUP……...32
5.1 Kesimpulan……...32
5.2 Saran…….32

DAFTAR PUSTAKA……...33

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi menuntut kemampuan dalam pemrograman, khususnya dalam pengolahan data. *JavaScript* menjadi salah satu bahasa yang banyak digunakan karena fleksibel dan mudah dipahami. Dalam *JavaScript,* penggunaan function membantu menyusun kode agar lebih terstruktur dan dapat digunakan kembali. Selain itu, array digunakan untuk menyimpan dan mengelola kumpulan data secara efisien. Pemahaman tentang *callback* dan *method array* seperti *map, filter,* dan *reduce* juga penting untuk memproses data dengan lebih cepat dan sederhana. Oleh karena itu, praktikum ini dilakukan untuk memahami dan menerapkan penggunaan function dan *array* dalam menyelesaikan masalah pemrograman.

Selain itu, pemahaman tentang konsep rekursi juga menjadi bagian penting dalam pemrograman karena dapat digunakan untuk menyelesaikan masalah secara bertahap dengan cara yang lebih efisien. Dengan menggabungkan *function, array,* callback, dan rekursi, mahasiswa dapat membangun logika program yang lebih terstruktur dan mudah dikembangkan. Praktikum ini membantu meningkatkan kemampuan berpikir logis dan pemecahan masalah melalui penerapan langsung dalam kode program.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, Maka rumusan masalah dalam laporan ini adalah sebagai berikut:

1. Bagaimana cara membuat dan menggunakan function dalam JavaScript?
2. Bagaimana cara menggunakan parameter dan return pada function?
3. Bagaimana cara menerapkan arrow function dan callback?
4. Bagaimana cara membuat dan mengelola data menggunakan array?
5. Bagaimana cara menggunakan method array seperti forEach, map, filter, dan reduce?
6. Bagaimana cara menerapkan rekursi dalam menyelesaikan masalah pemrograman?

1.3 Tujuan Praktikum

Tujuan praktikum ini adalah untuk memahami cara membuat dan menggunakan function dalam *JavaScript* serta menerapkan parameter dan return dalam proses pengolahan data. Selain itu, praktikum ini bertujuan untuk memahami penggunaan arrow function dan callback agar penulisan kode menjadi lebih efisien.

Selanjutnya, tujuan praktikum ini adalah untuk memahami cara mengelola data menggunakan array serta menerapkan method seperti *forEach, map, filter*, dan *reduce* dalam pengolahan data. Praktikum ini juga bertujuan untuk melatih kemampuan dalam mengolah data secara terstruktur.

Selain itu, praktikum ini bertujuan untuk memahami konsep rekursi dan menerapkannya dalam menyelesaikan masalah pemrograman. Dengan praktikum ini, diharapkan kemampuan berpikir logis dan pemecahan masalah dapat meningkat.

BAB II
DASAR TEORI

2.1 Function dalam JavaScript

Function merupakan sekumpulan kode yang digunakan untuk menjalankan suatu tugas tertentu dan dapat digunakan kembali. Dengan menggunakan function, program menjadi lebih terstruktur, rapi, dan mudah dipahami. Function dapat menerima input berupa parameter dan dapat mengembalikan nilai menggunakan return.

Dalam JavaScript, function dapat dibuat dengan beberapa cara, seperti function biasa dan arrow function. Function biasa ditulis menggunakan keyword function, sedangkan arrow function menggunakan sintaks yang lebih singkat. Selain itu, function juga dapat digunakan sebagai callback, yaitu function yang dikirim sebagai argumen ke function lain untuk dijalankan di dalamnya.

Penggunaan function sangat penting dalam pemrograman karena membantu mengurangi pengulangan kode dan mempermudah pengelolaan logika program. Dengan memahami function, programmer dapat membuat program yang lebih efisien dan mudah dikembangkan.

2.2 Array dalam JavaScript

Array merupakan struktur data yang digunakan untuk menyimpan sekumpulan nilai dalam satu variabel. Setiap data dalam array memiliki indeks yang dimulai dari 0, sehingga memudahkan dalam proses akses dan pengolahan data. Array dapat berisi berbagai jenis data seperti angka, string, maupun object.

Dalam JavaScript, array memiliki berbagai method yang memudahkan pengolahan data. Method seperti push dan pop digunakan untuk menambah dan menghapus data, sedangkan unshift dan shift digunakan untuk manipulasi di bagian awal array. Selain itu, terdapat method modern seperti forEach, map, filter, dan

reduce yang digunakan untuk melakukan iterasi, transformasi, penyaringan, dan pengolahan data menjadi satu nilai.

Penggunaan array sangat membantu dalam pengelolaan data dalam jumlah banyak. Dengan memahami array dan method yang tersedia, programmer dapat mengolah data dengan lebih cepat, efisien, dan terstruktur.

2.3 Callback dan Arrow Function

Callback merupakan function yang dikirim sebagai parameter ke dalam function lain untuk dijalankan di dalamnya. Konsep ini digunakan untuk membuat program lebih fleksibel karena function dapat menerima berbagai proses yang berbeda sesuai kebutuhan. Callback sering digunakan dalam pengolahan data dan operasi asynchronous seperti setTimeout.

Arrow function adalah bentuk penulisan function yang lebih singkat dalam JavaScript. Arrow function menggunakan tanda panah dan tidak memerlukan penulisan function secara lengkap, sehingga membuat kode lebih ringkas dan mudah dibaca. Arrow function juga sering digunakan bersama callback untuk membuat kode lebih sederhana.

Penggunaan callback dan arrow function membantu meningkatkan efisiensi penulisan kode dan mempermudah dalam pengolahan data. Dengan memahami konsep ini, programmer dapat membuat program yang lebih dinamis dan terstruktur.

2.4 Rekursi

Rekursi merupakan teknik pemrograman di mana sebuah function memanggil dirinya sendiri untuk menyelesaikan suatu masalah. Rekursi biasanya digunakan untuk masalah yang dapat dipecah menjadi bagian yang lebih kecil dan memiliki pola yang sama.

Dalam rekursi terdapat dua bagian penting, yaitu base case dan recursive case. Base case merupakan kondisi berhenti agar proses tidak berjalan terus-menerus, sedangkan recursive case adalah bagian di mana function memanggil dirinya sendiri dengan parameter yang lebih kecil atau mendekati kondisi berhenti.

Penggunaan rekursi dapat membantu menyederhanakan penyelesaian masalah tertentu seperti perhitungan faktorial, deret fibonacci, dan pengolahan data secara berulang. Dengan memahami rekursi, programmer dapat membuat solusi yang lebih efisien dan terstruktur.

BAB III
METODELOGI PRAKTIKUM

3.1 Waktu dan Tempat

Praktikum ini dilaksanakan pada tanggal 8 bertempat di laboratorium gedung jurusan Teknologi Informasi Komputer, Ruang 106. Kegiatan praktikum dilakukan sesuai dengan jadwal yang telah ditentukan.

3.2 Alat dan Bahan

Alat:

- Laptop atau computer
- Visual Studio Code atau text editor lainnya
- Node.js

Bahan:

- Modul praktikum Algoritma dan Struktur Data
- File latihan JavaScript
- Data percobaan untuk pengujian program

3.3 Prosedur Praktikum

Langkah langkah yang dilakukan pada praktikum adalah sebagai berikut.

1. Menyiapkan alat dan bahan yang dibutuhkan seperti laptop, text editor, dan Node.js.

2. Membuka Git Bash kemudian masuk ke repository 2025573010096_Algoritma_Dan_Struktur_Data menggunakan perintah cd.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~
$ cd 2025573010096_Algoritma_Dan_Struktur_Data
```

3. Membuat folder baru bernama praktikum-3 menggunakan perintah mkdir praktikum-3, kemudian masuk ke dalam folder tersebut dengan perintah cd praktikum-3. Setelah itu, melakukan inisialisasi proyek Node.js dengan menjalankan perintah npm init -y untuk membuat file package.json secara otomatis. Selanjutnya, membuka folder di Visual Studio Code menggunakan perintah code . agar dapat langsung melakukan penulisan kode.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~
$ cd 2025573010096_Algoritma_Dan_Struktur_Data

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data (
main)
$ mkdir praktikum-3

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data (
main)
$ cd praktikum-3

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/p
raktikum-3 (main)
$ npm init -y
Wrote to C:\Users\Lenovo LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum
-3\package.json:

{
  "name": "praktikum-3",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}


Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/p
raktikum-3 (main)
$ code .
```

4. Memastikan berada di dalam folder praktikum-3, kemudian membuat file baru bernama 01-function-dasar.js. Selanjutnya, menuliskan kode sesuai dengan modul ke dalam file tersebut. Kemudian Menambahkan kode baru sesuai dengan instruksi latihan. Kemudian membuat function kurang, kali, dan bagi, serta menambahkan pengecekan pada function bagi untuk kondisi pembagian dengan nol. Selanjutnya, membuat function kalkulator yang menerima tiga parameter dan menggunakan kondisi untuk menentukan operasi yang dijalankan. Setelah itu, melakukan pengujian function kalkulator dengan beberapa pemanggilan berbeda termasuk pembagian dengan nol, kemudian menyimpan file dan menjalankan program menggunakan perintah node 01-function-dasar.js untuk melihat hasil output.

JS 01-function-dasar.js U

Kode Program 01-function-dasar.js :

```
// 01-function-dasar.js
// ==================================================
// DASAR FUNCTION: CARA BUAT, INPUT, OUTPUT, DAN AREA VARIABEL
// ==================================================

// --- 1. Function tanpa input dan tanpa hasil balik ---
function sapa() {
    console.log('Halo! Selamat datang di praktikum JavaScript.');
}
sapa();     // menjalankan function
sapa();     // bisa dipanggil berulang kali

// --- 2. Function dengan input ---
function sapaNama(nama) {
    console.log(`Halo, ${nama}! Selamat belajar.`);
}
sapaNama('Budi');
sapaNama('Siti');
```

```
// --- 3. Function dengan input dan menghasilkan nilai ---
function tambah(angka1, angka2) {
    const hasil = angka1 + angka2;
    return hasil; // mengirim hasil keluar function
}

const hasilPenjumlahan = tambah(10, 25);
console.log('10 + 25 =', hasilPenjumlahan);
console.log('7 + 13 =', tambah(7, 13));     // langsung dipakai
tanpa simpan

// --- 4. Function dengan nilai awal otomatis ---
// jika tidak diberi nilai kedua, akan pakai nilai bawaan
function hitung(nilai, pengali = 2) {
    return nilai * pengali;
}
console.log(hitung(5));     // hasil pakai nilai default
console.log(hitung(5, 3));  // hasil pakai nilai yang dikirim

// --- 5. Area penggunaan variabel ---
const pesanGlobal = 'Saya ada di mana saja'; // bisa dipakai di mana
saja
function cekScope() {
    const pesanLokal = 'Saya hanya ada di dalam function ini'; //
hanya di dalam function
    console.log(pesanGlobal); // akses variabel luar
    console.log(pesanLokal); // akses variabel dalam
}

cekScope();
console.log(pesanGlobal);   // masih bisa dipakai di luar
// console.log(pesanLokal); // akan error jika diaktifkan

/* ============================
   BAGIAN LATIHAN: KALKULATOR
============================ */

// fungsi untuk pengurangan
function kurang(a, b) {
    return a - b;
}

// fungsi untuk perkalian
function kali(a, b) {
```

```
    return a * b;
}

// fungsi untuk pembagian dengan pengecekan
function bagi(a, b) {
    if (b === 0) {
        console.log('Error: tidak bisa membagi dengan nol');
        return null;
    }
    return a / b;
}

// fungsi utama untuk memilih operasi
function kalkulator(a, operator, b) {
    if (operator === '+') {
        return tambah(a, b);
    } else if (operator === '-') {
        return kurang(a, b);
    } else if (operator === '*') {
        return kali(a, b);
    } else if (operator === '/') {
        return bagi(a, b);
    } else {
        console.log('Operator tidak valid');
        return null;
    }
}

// percobaan hasil
console.log('10 + 5 =', kalkulator(10, '+', 5));
console.log('10 - 5 =', kalkulator(10, '-', 5));
console.log('10 * 5 =', kalkulator(10, '*', 5));
console.log('10 / 5 =', kalkulator(10, '/', 5));
console.log('10 / 0 =', kalkulator(10, '/', 0));
```

Hasil output :

```
[Running] node "c:\Users\Lenovo
LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-
3\tempCodeRunnerFile.js"
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
```

```
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer

=== LATIHAN CALLBACK ===
Hasil: BELAJAR JAVASCRIPT ITU MENYENANGKAN
Hasil: belajar javascript itu menyenangkan!!!
Hasil: 4
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)

[Done] exited with code=0 in 1.314 seconds
```

5. Buat file baru bernama 02-arrow-callback.js menggunakan perintah touch di terminal. Setelah file dibuat, buka file tersebut dan ketik kode yang ada di modul sesuai instruksi. Setelah itu, tambahkan kode latihan yang diberikan di bawah kode utama sesuai petunjuk pada modul. Simpan file setelah selesai. Jalankan file di terminal menggunakan perintah node 02-arrow-callback.js. Tampilkan kembali kode yang telah diketik beserta hasil output yang muncul di terminal sebagai bagian dari laporan praktikum.

JS 02-arrow-callback.js

Kode Program 02-arrow-callback.js :

```
// 02-arrow-callback.js

// ================================================
// PENULISAN FUNCTION MODERN DAN PENGGUNAAN CALLBACK
// ================================================
```

```
// --- 1. Perbandingan cara membuat function ---
// metode lama
function kuadratBiasa(x) {
    return x * x;
}

// metode modern dengan arrow
const kuadrat = (x) => {
    return x * x;
};

// versi paling singkat
const kuadratRingkas = x => x * x;

console.log('=== Perbandingan Penulisan ===');
console.log('Biasa :', kuadratBiasa(5));        // hasil kuadrat
console.log('Arrow :', kuadrat(5));             // hasil sama
console.log('Ringkas :', kuadratRingkas(5));    // hasil sama

// --- 2. Arrow function dengan lebih dari satu input ---
const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;

console.log('Luas 4x6 :', luas(4, 6));
console.log(salam('Budi', 'Pagi'));

// --- 3. Function yang menerima function lain ---
function lakukanOperasi(angka, operasiCallback) {
    console.log(`Angka awal: ${angka}`);
    const hasil = operasiCallback(angka);
    console.log(`Hasil setelah operasi: ${hasil}`);
}

console.log('\n=== Callback ===');
lakukanOperasi(7, kuadratRingkas);       // kirim fungsi kuadrat
lakukanOperasi(10, x => x + 100);        // kirim fungsi langsung
lakukanOperasi(20, function(x) {         // kirim fungsi biasa
return x / 2;
});

// --- 4. Contoh penggunaan callback dengan timer ---
console.log('\n=== setTimeout (Callback) ===');
console.log('Pesan 1: Sebelum timer');

setTimeout(() => {
```

```
    console.log('Pesan 3: Ini dari dalam setTimeout (setelah 1
detik)');
}, 1000);   // jeda 1 detik

console.log('Pesan 2: Setelah mendaftarkan timer');

// ============================
// LATIHAN 2: OLAH KALIMAT DENGAN CALLBACK
// ============================

// ubah semua huruf jadi kapital
const keHurufBesar = (teks) => teks.toUpperCase();

// tambah tanda seru di akhir
const tambahSeru = (teks) => teks + '!!!';

// hitung jumlah kata dalam kalimat
const hitungKata = (teks) => teks.split(' ').length;

// jalankan transformasi dengan callback
function prosesKalimat(kalimat, transformasiCallback) {
    const hasil = transformasiCallback(kalimat);
    console.log('Hasil:', hasil);
}

// percobaan hasil
console.log('\n=== LATIHAN CALLBACK ===');

const kalimat = 'belajar javascript itu menyenangkan';

prosesKalimat(kalimat, keHurufBesar);
prosesKalimat(kalimat, tambahSeru);
prosesKalimat(kalimat, hitungKata);
```

Hasil output :

```
[Running] node "c:\Users\Lenovo
LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-
3\tempCodeRunnerFile.js"
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
```

```
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer

=== LATIHAN CALLBACK ===
Hasil: BELAJAR JAVASCRIPT ITU MENYENANGKAN
Hasil: belajar javascript itu menyenangkan!!!
Hasil: 4
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)

[Done] exited with code=0 in 1.314 seconds
```

6. Buat file baru bernama 03-array-dasar.js menggunakan perintah touch di terminal. Setelah file berhasil dibuat, buka file tersebut dan ketik kode yang ada di modul sesuai instruksi. Lanjutkan dengan menambahkan kode latihan array yang diberikan di bawah kode utama sesuai petunjuk modul. Simpan file setelah selesai. Jalankan file di terminal menggunakan perintah node 03-array-dasar.js. Tampilkan kode yang sudah diketik beserta hasil output yang muncul di terminal sebagai bagian dari laporan praktikum.

JS 03-array-dasar.js

Kode Program 03-array-dasar.js :

```
// 03-array-dasar.js
// ================================================
// PENGGUNAAN ARRAY: BUAT, AKSES, DAN UBAH DATA
// ================================================

// --- 1. Inisialisasi array ---
```

```
const mahasiswa = ['Budi', 'Siti', 'Ahmad', 'Rina'];
const nilai = [85, 92, 78, 95, 88];
console.log('=== Array Awal ===');
console.log('Mahasiswa          :', mahasiswa);
console.log('Nilai              :', nilai);
console.log('Jumlah mahasiswa   :', mahasiswa.length);

// --- 2. Mengambil data berdasarkan posisi ---
console.log('\n=== Akses Elemen ===');
console.log('Elemen pertama     :', mahasiswa[0]);      // posisi
awal
console.log('Elemen ketiga      :', mahasiswa[2]);      // posisi
ke-3
console.log('Elemen terakhir    :', mahasiswa[mahasiswa.length -
1]);

// --- 3. Mengganti isi array ---
mahasiswa[1] = 'Siti Rahayu';           // ubah data pada posisi
tertentu
console.log('\nSetelah diubah:', mahasiswa);

// --- 4. Operasi tambah dan hapus ---
console.log('\n=== Manipulasi Array ===');
mahasiswa.push('Doni');                 // tambah data di belakang
console.log('Setelah push       :', mahasiswa);
mahasiswa.unshift('Zahra');             // tambah data di depan
console.log('Setelah unshift    :', mahasiswa);
const dihapusAkhir = mahasiswa.pop();   // hapus data terakhir
console.log('Dihapus (pop)      :', dihapusAkhir);
console.log('Setelah pop        :', mahasiswa);
const dihapusAwal = mahasiswa.shift();  // hapus data pertama
console.log('Dihapus (shift)    :', dihapusAwal);
console.log('Setelah shift      :', mahasiswa);

// --- 5. Mencari data dalam array ---
console.log('\n=== Pencarian ===');
console.log('Indeks Ahmad       :', mahasiswa.indexOf('Ahmad'));
console.log('Ada Rina?          :', mahasiswa.includes('Rina'));
console.log('Ada Budi?          :', mahasiswa.includes('Budi'));

// --- 6. Mengambil sebagian isi array ---
const sebagian = nilai.slice(1, 4); // ambil dari indeks 1 sampai
sebelum 4
console.log('\nNilai asli       :', nilai);
console.log('Slice [1,4]        :', sebagian);
```

```
// ============================
// LATIHAN 3: LIST BELANJA
// ============================

// daftar awal barang
let daftarBelanja = ['Beras', 'Gula', 'Minyak', 'Telur', 'Garam'];

// tampilkan semua item dengan nomor urut
console.log('\n=== DAFTAR BELANJA AWAL ===');
for (let i = 0; i < daftarBelanja.length; i++) {
    console.log((i + 1) + '. ' + daftarBelanja[i]);
}

// tambah item baru di bagian akhir
daftarBelanja.push('Susu');
daftarBelanja.push('Kopi');

console.log('\nSetelah tambah item:', daftarBelanja);

// hapus item pertama dari daftar
const itemDihapus = daftarBelanja.shift();
console.log('\nItem yang dihapus:', itemDihapus);

// cek keberadaan item tertentu
const adaSusu = daftarBelanja.includes('Susu');
console.log('\nApakah ada Susu?', adaSusu ? 'Ya, ada' : 'Tidak
ada');

// tampilkan total item terbaru
console.log('\nJumlah item sekarang:', daftarBelanja.length);
```

Hasil output :

```
[Running] node "c:\Users\Lenovo
LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-
3\tempCodeRunnerFile.js"
=== Array Awal ===
Mahasiswa          : [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai              : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa   : 4

=== Akses Elemen ===
Elemen pertama     : Budi
Elemen ketiga      : Ahmad
```

```
Elemen terakhir    : Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push       : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina',
'Doni' ]
Setelah unshift    : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad',
'Rina', 'Doni' ]
Dihapus (pop)      : Doni
Setelah pop        : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad',
'Rina' ]
Dihapus (shift)    : Zahra
Setelah shift      : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad       : 2
Ada Rina?          : true
Ada Budi?          : true

Nilai asli       : [ 85, 92, 78, 95, 88 ]
Slice [1,4]        : [ 92, 78, 95 ]

=== DAFTAR BELANJA AWAL ===
1. Beras
2. Gula
3. Minyak
4. Telur
5. Garam

Setelah tambah item: [
  'Beras',  'Gula',
  'Minyak', 'Telur',
  'Garam',  'Susu',
  'Kopi'
]

Item yang dihapus: Beras

Apakah ada Susu? Ya, ada

Jumlah item sekarang: 6

[Done] exited with code=0 in 0.248 seconds
```

7. Buat file baru bernama 04-array-method.js menggunakan perintah touch di terminal. Setelah file dibuat, buka file tersebut dan ketik kode yang ada di modul sesuai instruksi. Setelah itu, tambahkan kode latihan array method yang diberikan di bawah kode utama sesuai petunjuk modul. Simpan file setelah selesai. Jalankan file di terminal menggunakan perintah node 04-array-method.js. Tampilkan kode yang sudah diketik beserta hasil output yang muncul di terminal sebagai bagian dari laporan praktikum.

JS 04-array-method.js

Kode Program 04-array-method.js :

```
// 04-array-method.js
// ================================================
// PENGULANGAN ARRAY MODERN: forEach, map, filter, reduce
// ================================================

const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];

// --- 1. forEach: menjalankan aksi untuk setiap data ---
console.log('=== forEach: Tampilkan Semua Nilai ===');
nilaiMahasiswa.forEach((nilai, indeks) => {
    console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});

// --- 2. map: mengubah isi array menjadi bentuk baru ---
// ubah nilai angka menjadi huruf
const gradeHuruf = nilaiMahasiswa.map(nilai => {
    if (nilai >= 90) return 'A';
    if (nilai >= 80) return 'B';
    if (nilai >= 70) return 'C';
    if (nilai >= 60) return 'D';
    return 'E';
});

console.log('\n=== map: Nilai ke Grade ===');
console.log('Nilai :', nilaiMahasiswa);
console.log('Grade :', gradeHuruf);

// --- 3. filter: mengambil data sesuai syarat ---
const nilaiLulus = nilaiMahasiswa.filter(nilai => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter(nilai => nilai < 60);
```

```
console.log('\n=== filter: Lulus dan Tidak Lulus ===');
console.log('Semua nilai :', nilaiMahasiswa);
console.log('Nilai lulus :', nilaiLulus);
console.log('Nilai gagal :', nilaiGagal);

// --- 4. reduce: menjumlahkan semua nilai ---
const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
    return akumulator + nilai;
}, 0);      // nilai awal penjumlahan

const rataRata = totalNilai / nilaiMahasiswa.length;
console.log('\n=== reduce: Statistik Nilai ===');
console.log('Total nilai :', totalNilai);
console.log('Rata-rata :', rataRata.toFixed(2));

// --- 5. gabungkan beberapa method sekaligus ---
const rataRataNilaiLulus = nilaiMahasiswa
.filter(nilai => nilai >= 60)   // ambil nilai yang lolos
.reduce((acc, val, idx, arr) => {
return acc + val / arr.length;  // hitung rata-rata langsung
}, 0);

console.log('\n=== Method Chaining ===');
console.log('Rata-rata nilai lulus:',
rataRataNilaiLulus.toFixed(2));

// ============================
// LATIHAN 4: DATA PRODUK
// ============================

// daftar produk
const produk = [
    { nama: 'Laptop', harga: 8500000, stok: 5 },
    { nama: 'Mouse', harga: 150000, stok: 0 },
    { nama: 'Keyboard', harga: 450000, stok: 12 },
    { nama: 'Monitor', harga: 3200000, stok: 0 },
    { nama: 'Headset', harga: 350000, stok: 8 }
];

// ambil produk yang masih ada stok
const produkTersedia = produk.filter(p => p.stok > 0);
console.log('\n=== Produk Tersedia ===');
console.log(produkTersedia);

// ambil nama produk saja
```

```
const namaProduk = produkTersedia.map(p => p.nama);
console.log('\nNama Produk Tersedia:', namaProduk);

// hitung total nilai semua barang
const totalInventaris = produkTersedia.reduce((total, p) => {
    return total + (p.harga * p.stok);
}, 0);

console.log('\nTotal Nilai Inventaris:', totalInventaris);

// tampilkan semua produk dengan status
console.log('\n=== Daftar Produk ===');
produk.forEach(p => {
    if (p.stok > 0) {
        console.log(`[TERSEDIA] ${p.nama} - Rp${p.harga} (${p.stok}
unit)`);
    } else {
        console.log(`[HABIS] ${p.nama} - Rp${p.harga}`);
    }
});
```

Hasil output :

```
[Running] node "c:\Users\Lenovo
LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-
3\tempCodeRunnerFile.js"
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
```

```
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]

=== reduce: Statistik Nilai ===
Total nilai : 590
Rata-rata : 73.75

=== Method Chaining ===
Rata-rata nilai lulus: 81.17

=== Produk Tersedia ===
[
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]

Nama Produk Tersedia: [ 'Laptop', 'Keyboard', 'Headset' ]

Total Nilai Inventaris: 50700000

=== Daftar Produk ===
[TERSEDIA] Laptop - Rp8500000 (5 unit)
[HABIS] Mouse - Rp150000
[TERSEDIA] Keyboard - Rp450000 (12 unit)
[HABIS] Monitor - Rp3200000
[TERSEDIA] Headset - Rp350000 (8 unit)

[Done] exited with code=0 in 0.236 seconds
```

8. Buat file baru bernama 05-rekursi.js menggunakan perintah touch di terminal. Setelah file dibuat, buka file tersebut dan ketik kode yang ada di modul sesuai instruksi. Simpan file setelah selesai. Jalankan file di terminal menggunakan perintah node 05-rekursi.js. Tampilkan kode yang sudah diketik beserta hasil output yang muncul di terminal sebagai bagian dari laporan praktikum.

JS 05-rekursi.js

Kode Program 04-array-method.js :

```
// 05-rekursi.js
// ================================================
// FUNCTION BERULANG: PEMANGGILAN DIRI SENDIRI
// ================================================

// --- 1. Menghitung faktorial ---
function faktorial(n) {
    // kondisi berhenti
    if (n <= 1) return 1;
    // proses berulang
    return n * faktorial(n - 1);
}

console.log('=== Faktorial ===');
console.log('0! =', faktorial(0));  // hasil standar
console.log('1! =', faktorial(1));  // hasil dasar
console.log('5! =', faktorial(5));  // perhitungan bertahap
console.log('7! =', faktorial(7));  // hasil lebih besar

// --- 2. Deret fibonacci ---
// pola angka bertambah dari dua angka sebelumnya
function fibonacci(n) {
    // kondisi awal
    if (n === 0) return 0;
    if (n === 1) return 1;

    // proses hitung berulang
    return fibonacci(n - 1) + fibonacci(n - 2);
}

console.log('\n=== Fibonacci ===');
```

```
for (let i = 0; i <= 8; i++) {
    process.stdout.write(fibonacci(i) + ' ');
}
console.log(''); // pindah baris

// --- 3. Menjumlahkan isi array dengan rekursi ---
function jumlahArray(arr, indeks = 0) {
    // berhenti saat data habis
    if (indeks === arr.length) return 0;

    // tambah nilai satu per satu
    return arr[indeks] + jumlahArray(arr, indeks + 1);
}

const angka = [3, 7, 2, 9, 5];
console.log('\n=== Jumlah Array dengan Rekursi ===');
console.log('Array :', angka);
console.log('Jumlah :', jumlahArray(angka)); // total hasil

// --- 4. Hitung mundur ---
function countdown(n) {
    if (n < 0) {
        console.log('Selesai!');
        return;
}

console.log(n);
countdown(n - 1);
}

console.log('\n=== Countdown Rekursif ===');
countdown(5);
```

Hasil output :

```
[Running] node "c:\Users\Lenovo
LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-3\05-
rekursi.js"
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040
```

```
=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!

[Done] exited with code=0 in 0.262 seconds
```

9. Buat folder baru bernama tugas didalam folder praktikum-3

10. Buat file baru bernama tugas-1.js di dalam folder tugas. Kerjakan tugas sesuai modul dengan membuat array data mahasiswa, lalu membuat function untuk menghitung statistik nilai, memfilter mahasiswa yang lulus, dan menambahkan grade pada setiap data menggunakan reduce, filter, dan map. Ketik kode sesuai instruksi modul ke dalam file, simpan file, lalu jalankan di terminal menggunakan perintah node tugas/tugas-1.js. Tampilkan kode yang sudah dibuat beserta hasil output di terminal sebagai bagian laporan praktikum.

Kode Program tugas-1.js :

```
// ============================
// TUGAS 1: PENGOLAHAN DATA NILAI
// ============================

// kumpulan data mahasiswa
const dataMahasiswa = [
    { nama: 'Budi', nilai: 85 },
    { nama: 'Siti', nilai: 92 },
    { nama: 'Ahmad', nilai: 76 },
    { nama: 'Rina', nilai: 60 },
    { nama: 'Doni', nilai: 55 },
    { nama: 'Lina', nilai: 88 }
];

// fungsi untuk menghitung ringkasan nilai
function hitungStatistik(arr) {
    const hasil = arr.reduce((acc, mhs) => {
        acc.total++;
        acc.jumlahNilai += mhs.nilai;

        if (mhs.nilai > acc.tertinggi) acc.tertinggi = mhs.nilai;
        if (mhs.nilai < acc.terendah) acc.terendah = mhs.nilai;

        return acc;
    }, {
        total: 0,
        jumlahNilai: 0,
        tertinggi: -Infinity,
        terendah: Infinity
    });

    return {
        total: hasil.total,
        rataRata: hasil.jumlahNilai / hasil.total,
        tertinggi: hasil.tertinggi,
        terendah: hasil.terendah
    };
}

// ambil mahasiswa yang memenuhi syarat kelulusan
function filterLulus(arr, batasLulus) {
    return arr.filter(mhs => mhs.nilai >= batasLulus);
}
```

```
// tambahkan informasi grade ke setiap data
function tambahGrade(arr) {
    return arr.map(mhs => {
        let grade;

        if (mhs.nilai >= 90) grade = 'A';
        else if (mhs.nilai >= 80) grade = 'B';
        else if (mhs.nilai >= 70) grade = 'C';
        else if (mhs.nilai >= 60) grade = 'D';
        else grade = 'E';

        return { ...mhs, grade };
    });
}

// tampilkan semua hasil pengolahan
console.log('=== STATISTIK ===');
const statistik = hitungStatistik(dataMahasiswa);
console.log(`Total: ${statistik.total}`);
console.log(`Rata-rata: ${statistik.rataRata.toFixed(2)}`);
console.log(`Tertinggi: ${statistik.tertinggi}`);
console.log(`Terendah: ${statistik.terendah}`);

console.log('\n=== MAHASISWA LULUS (>= 70) ===');
const lulus = filterLulus(dataMahasiswa, 70);
lulus.forEach(mhs => {
    console.log(`${mhs.nama} - ${mhs.nilai}`);
});

console.log('\n=== DATA DENGAN GRADE ===');
const denganGrade = tambahGrade(dataMahasiswa);
denganGrade.forEach(mhs => {
    console.log(`${mhs.nama} - ${mhs.nilai} (${mhs.grade})`);
});
```

Hasil output :

```
[Running] node "c:\Users\Lenovo
LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-
3\tugas\tempCodeRunnerFile.js"
=== STATISTIK ===
Total: 6
Rata-rata: 76.00
Tertinggi: 92
```

```
Terendah: 55

=== MAHASISWA LULUS (>= 70) ===
Budi - 85
Siti - 92
Ahmad - 76
Lina - 88

=== DATA DENGAN GRADE ===
Budi - 85 (B)
Siti - 92 (A)
Ahmad - 76 (C)
Rina - 60 (D)
Doni - 55 (E)
Lina - 88 (B)

[Done] exited with code=0 in 0.372 seconds
```

11. Buat file baru bernama tugas-2.js di dalam folder tugas. Kerjakan tugas sesuai modul yang berisi pengolahan data menggunakan array dan function JavaScript seperti map, filter, reduce, atau method lain sesuai instruksi. Ketik kode sesuai petunjuk modul ke dalam file, simpan file setelah selesai, lalu jalankan di terminal menggunakan perintah node tugas/tugas-2.js. Tampilkan kode yang sudah dibuat beserta hasil output di terminal sebagai bagian laporan praktikum.

JS tugas-2.js

Kode Program tugas-2.js :

```
// ============================
// TUGAS 2: FUNCTION BERULANG
// ============================

// menghitung perpangkatan tanpa operator bawaan
function pangkat(basis, eksponen) {
    if (eksponen === 0) return 1; // kondisi berhenti
    return basis * pangkat(basis, eksponen - 1); // proses berulang
```

```
}

// membalik urutan karakter dalam string
function balikString(str) {
    if (str.length <= 1) return str; // kondisi akhir
    return str[str.length - 1] + balikString(str.slice(0, str.length
- 1));
}

// mengecek apakah kata sama saat dibalik
function cekPalindrom(str) {
    const hasilBalik = balikString(str);
    return str === hasilBalik;
}

// ============================
// UJI HASIL
// ============================

console.log('=== PANGKAT ===');
console.log('2^3 =', pangkat(2, 3));
console.log('5^2 =', pangkat(5, 2));
console.log('3^4 =', pangkat(3, 4));

console.log('\n=== BALIK STRING ===');
console.log('halo ->', balikString('halo'));
console.log('javascript ->', balikString('javascript'));

console.log('\n=== CEK PALINDROM ===');
const kata = ['katak', 'civic', 'level', 'halo'];

kata.forEach(k => {
    const hasil = cekPalindrom(k) ? 'Palindrom' : 'Bukan palindrom';
    console.log(`${k} -> ${hasil}`);
});
```

Hasil output :

```
[Running] node "c:\Users\Lenovo
LOQ\2025573010096_Algoritma_Dan_Struktur_Data\praktikum-
3\tugas\tugas-2.js"
=== PANGKAT ===
2^3 = 8
5^2 = 25
```

```
3^4 = 81

=== BALIK STRING ===
halo -> olah
javascript -> tpircsavaj

=== CEK PALINDROM ===
katak -> Palindrom
civic -> Palindrom
level -> Palindrom
halo -> Bukan palindrom

[Done] exited with code=0 in 0.274 seconds
```

12. Membuka Git Bash melalui terminal, Pastikan berada di praktikum-3, kemudian menjalankan perintah git add ., Untuk menambahkan semua file ke dalam staging area.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$ git add .
warning: in the working copy of 'praktikum-3/package.json', LF will be replaced by CRLF the next time Git touches it

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$
```

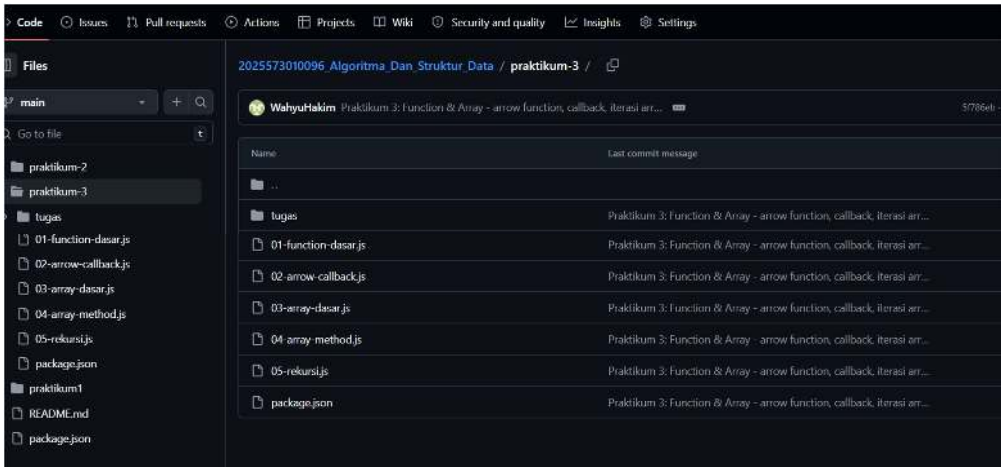
13. Melakukan commit dengan menjalankan perintah git commit -m "Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi"

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$ git commit -m "Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi"
[main 5f786eb] Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi
 8 files changed, 542 insertions(+)
 create mode 100644 praktikum-3/01-function-dasar.js
 create mode 100644 praktikum-3/02-arrow-callback.js
 create mode 100644 praktikum-3/03-array-dasar.js
 create mode 100644 praktikum-3/04-array-method.js
 create mode 100644 praktikum-3/05-rekursi.js
 create mode 100644 praktikum-3/package.json
 create mode 100644 praktikum-3/tugas/tugas-1.js
 create mode 100644 praktikum-3/tugas/tugas-2.js

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$
```

14. Mengirim file ke repositori GitHub dengan menjalankan perintah git push.

```
Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 6.80 KiB | 994.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/WahyuHakim/2025573010096_Algoritma_Dan_Struktur_Data.git
   cbd8efb..5f786eb  main -> main

Lenovo LOQ@LAPTOP-UJADEQMH MINGW64 ~/2025573010096_Algoritma_Dan_Struktur_Data/praktikum-3 (main)
$
```

15. Langkah terakhir adalah membuka GitHub melalui browser, kemudian masuk ke akun GitHub untuk mengakses repositori yang telah dibuat sebagai tempat penyimpanan file praktikum.



BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil praktikum

Hasil praktikum menunjukkan seluruh proses pembuatan dan eksekusi file JavaScript berjalan sesuai instruksi modul. Mulai dari pembuatan file menggunakan perintah touch, penulisan kode function dasar, arrow function, callback, hingga pengolahan array menggunakan map, filter, dan reduce. Setiap file berhasil dijalankan menggunakan Node.js di terminal dan menghasilkan output sesuai fungsi yang dibuat seperti hasil perhitungan operasi dasar, manipulasi array, dan pengolahan data mahasiswa. Pembuatan folder tugas juga membantu dalam mengorganisasi file agar lebih rapi dan mudah dikelola dalam satu struktur proyek.

4.2 Pembahasan

Proses praktikum ini melatih pemahaman alur kerja pemrograman JavaScript dari tahap penulisan kode, penyimpanan file, hingga eksekusi di terminal. Penggunaan function membantu memahami konsep dasar logika pemrograman, sedangkan array method seperti map, filter, dan reduce memperkuat kemampuan pengolahan data. Selain itu, penggunaan Git dalam proses add juga memperkenalkan sistem version control, meskipun sempat muncul kendala seperti warning CRLF dan kesalahan path, namun hal tersebut dapat dipahami sebagai bagian dari perbedaan sistem operasi dan struktur folder, sehingga tidak mengganggu hasil akhir praktikum secara keseluruhan.

BAB V
PENUTUP

5.1 Kesimpulan

Praktikum ini berhasil membantu memahami dasar pemrograman JavaScript mulai dari function, arrow function, callback, hingga pengolahan array menggunakan map, filter, dan reduce. Seluruh file dapat dijalankan dengan Node.js dan menghasilkan output sesuai dengan logika yang dibuat. Praktikum ini juga melatih alur kerja dasar pengembangan program dari penulisan kode, eksekusi, hingga pengelolaan file menggunakan folder dan Git.

5.2 Saran

Perlu lebih sering latihan membuat function dan array method agar lebih cepat memahami logika pemrograman. Gunakan struktur folder yang rapi sejak awal agar tidak terjadi kesalahan path saat menjalankan file. Pahami juga perintah Git dasar seperti add, commit, dan push supaya proses pengumpulan tugas lebih lancar..

DAFTAR PUSTAKA

MDN Web Docs. JavaScript Guide.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Node.js Foundation. Node.js Documentation.
https://nodejs.org/en/docs

W3Schools. JavaScript Tutorial.
https://www.w3schools.com/js/

Mozilla Foundation. Array Methods in JavaScript.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

Git Documentation. Git Reference Manual.
https://git-scm.com/doc